

DASTUR KODI

«Maktab Xabarnoma» dasturining asosiy funksiyalarini bajaruvchi modullar

Sana: 07.07.2026

Dasturlash tili: Python 3.12

Framework: FastAPI, SQLAlchemy, python-telegram-bot

Ushbu hujjat dasturning to'liq kod bazasining bir qismini — asosiy biznes-logika, modellar, API, servislar va Telegram bot handlerlarini o'z ichiga oladi.

MUNDARIJA

1. start.py — Dasturni ishga tushirish
2. app/main.py — FastAPI ilovasi
3. app/config.py — Sozlamalar
4. app/database.py — Ma'lumotlar bazasi
5. app/models/student.py — O'quvchi modeli
6. app/models/parent.py — Ota-ona modeli
7. app/models/device.py — Turniket modeli
8. app/models/attendance.py — Davomat modeli
9. app/models/settings.py — Tizim sozlamalari
10. app/models/payment.py — To'lov buyurtmasi
11. app/schemas/__init__.py — API sxemalari (qism)
12. app/api/webhooks.py — Turniket webhook
13. app/services/attendance.py — Davomat servisi
14. app/services/telegram.py — Telegram xabar servisi
15. app/services/subscription.py — Obuna servisi
16. app/services/payment.py — To'lov servisi
17. app/bot/handlers.py — Telegram bot buyruqlari
18. app/api/payments.py — To'lov sahifalari
19. app/api/admin.py — Admin API (qism)

TIZIM ARXITEKTURASI (BLOK-SXEMA)

[Turniket / Face ID terminali]

| HTTP POST (card_id, device_code, direction)

v

[FastAPI Server — /webhooks/turnstile]

|

v

[AttendanceService.process()]

1. O'quvchini card_id bo'yicha topish
2. Turniketni device_code bo'yicha topish
3. AttendanceEvent yozuvi yaratish
4. Ota-onalarni StudentParent orqali topish
5. SubscriptionService — obuna tekshiruvi
6. TelegramService — xabar yuborish

|

v

[Telegram Bot — ota-ona telefonida xabar]

Parallel jarayon: Telegram Bot (register, pay, status) <-> SQLite DB <-> Admin Panel

start.py — Dasturni ishga tushirish

start.py

```
#!/usr/bin/env python3
"""API + Bot ishga tushirish."""
import argparse, multiprocessing, sys, os
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

def api():
    import uvicorn
    uvicorn.run("app.main:app", host="0.0.0.0", port=8000)

def bot():
    from app.bot.handlers import run_bot
    run_bot()

if __name__ == "__main__":
    p = argparse.ArgumentParser()
    p.add_argument("--api", action="store_true")
    p.add_argument("--bot", action="store_true")
    a = p.parse_args()
    if a.api:
        print(" http://localhost:8000/admin")
        api()
    elif a.bot:
        bot()
    else:
        print("=" * 40)
        print("  Maktab Xabarnoma v3.0")
        print("  Admin: http://localhost:8000/admin")
        print("=" * 40)
        pa = multiprocessing.Process(target=api)
        pb = multiprocessing.Process(target=bot)
        pa.start(); pb.start()
        pa.join(); pb.join()
```

app/main.py — FastAPI ilovasi

app/main.py

```

from contextlib import asynccontextmanager
from pathlib import Path

from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import Response

from app.api.admin import router as admin_router
from app.api.payments import router as pay_router, wh as pay_wh
from app.api.webhooks import router as wh_router
from app.config import settings
from app.database import init_db

STATIC = Path(__file__).parent / "static"

@asynccontextmanager
async def lifespan(app: FastAPI):
    await init_db()
    yield

app = FastAPI(title=settings.app_name, version="3.0.0", lifespan=lifespan)
app.add_middleware(CORSMiddleware, allow_origins=["*"], allow_methods=["*"], allow_headers=["*"])
app.include_router(wh_router)
app.include_router(pay_wh)
app.include_router(pay_router)
app.include_router(admin_router)

@app.get("/")
def root():
    return {"app": settings.app_name, "version": "3.0", "admin": "/admin"}

@app.get("/admin")
def admin():
    html = (STATIC / "admin.html").read_text(encoding="utf-8")
    return Response(html, media_type="text/html", headers={"Cache-Control": "no-store"})

@app.get("/health")
def health():
    return {"ok": True, "telegram": bool(settings.telegram_bot_token)}

```

app/config.py — Sozlamalar

app/config.py

```
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", env_file_encoding="utf-8", extra="ignore")

    app_name: str = "Maktab Xabarnoma"
    database_url: str = "sqlite+aiosqlite:///./xabarnoma.db"
    telegram_bot_token: str = ""
    webhook_secret: str = "change-me-in-production"
    admin_api_key: str = "admin-secret-key"
    timezone: str = "Asia/Tashkent"
    app_base_url: str = "http://localhost:8000"
    payment_demo_mode: bool = True
    payment_webhook_secret: str = "payment-secret"
    click_service_id: str = ""
    click_merchant_id: str = ""
    click_secret_key: str = ""

settings = Settings()
```

app/database.py — Ma'lumotlar bazasi

app/database.py

```

from collections.abc import AsyncGenerator

from sqlalchemy.ext.asyncio import AsyncSession, async_sessionmaker, create_async_engine
from sqlalchemy.orm import DeclarativeBase

from app.config import settings

class Base(DeclarativeBase):
    pass

engine = create_async_engine(settings.database_url, echo=False)
SessionLocal = async_sessionmaker(engine, class_=AsyncSession, expire_on_commit=False)

async def get_db() -> AsyncGenerator[AsyncSession, None]:
    async with SessionLocal() as session:
        yield session

async def _migrate_columns(conn) -> None:
    """Mavjud SQLite bazaga yangi ustunlar qo'shish."""
    from sqlalchemy import inspect, text

    def _run(sync_conn):
        insp = inspect(sync_conn)
        if "settings" not in insp.get_table_names():
            return
        cols = {c["name"] for c in insp.get_columns("settings")}
        for name, ddl in (
            ("payment_card_number", "ALTER TABLE settings ADD COLUMN payment_card_number VARCHAR(30)"),
            ("payment_card_holder", "ALTER TABLE settings ADD COLUMN payment_card_holder VARCHAR(100)"),
            ("payment_click_link", "ALTER TABLE settings ADD COLUMN payment_click_link TEXT"),
        ):
            if name not in cols:
                sync_conn.execute(text(ddl))

    await conn.run_sync(_run)

async def init_db() -> None:
    from app import models # noqa: F401

    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.create_all)
        await _migrate_columns(conn)

```

app/models/student.py — O'quvchi modeli

app/models/student.py

```
from datetime import datetime

from sqlalchemy import DateTime, ForeignKey, String, func
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.database import Base


class Student(Base):
    __tablename__ = "students"

    id: Mapped[int] = mapped_column(primary_key=True)
    school_id: Mapped[int] = mapped_column(ForeignKey("schools.id"))
    first_name: Mapped[str] = mapped_column(String(100))
    last_name: Mapped[str] = mapped_column(String(100))
    class_name: Mapped[str] = mapped_column(String(50))
    card_id: Mapped[str] = mapped_column(String(64), unique=True, index=True)
    is_active: Mapped[bool] = mapped_column(default=True)
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), server_default=func.now())

    school: Mapped["School"] = relationship(back_populates="students")
    parents: Mapped[list["Parent"]] = relationship(secondary="student_parents", back_populates="students")

from app.models.parent import Parent # noqa: E402
from app.models.school import School # noqa: E402
```

app/models/parent.py — Ota-ona modeli

app/models/parent.py

```

from datetime import datetime

from sqlalchemy import BigInteger, DateTime, ForeignKey, String, func
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.database import Base


class Parent(Base):
    __tablename__ = "parents"

    id: Mapped[int] = mapped_column(primary_key=True)
    full_name: Mapped[str] = mapped_column(String(200))
    phone: Mapped[str | None] = mapped_column(String(20))
    telegram_chat_id: Mapped[int | None] = mapped_column(BigInteger, unique=True, index=True)
    telegram_username: Mapped[str | None] = mapped_column(String(100))
    bot_registered_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True))
    subscription_until: Mapped[datetime | None] = mapped_column(DateTime(timezone=True))
    is_active: Mapped[bool] = mapped_column(default=True)
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), server_default=func.now())

    students: Mapped[list["Student"]] = relationship(secondary="student_parents", back_populates="parents")


class StudentParent(Base):
    __tablename__ = "student_parents"

    id: Mapped[int] = mapped_column(primary_key=True)
    student_id: Mapped[int] = mapped_column(ForeignKey("students.id"))
    parent_id: Mapped[int] = mapped_column(ForeignKey("parents.id"))
    relation: Mapped[str] = mapped_column(String(50), default="ota-ona")


from app.models.student import Student # noqa: E402

```


app/models/device.py — Turniket modeli

app/models/device.py

```
from datetime import datetime

from sqlalchemy import DateTime, ForeignKey, String, func
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.database import Base


class Device(Base):
    __tablename__ = "devices"

    id: Mapped[int] = mapped_column(primary_key=True)
    school_id: Mapped[int] = mapped_column(ForeignKey("schools.id"))
    device_code: Mapped[str] = mapped_column(String(64), unique=True, index=True)
    name: Mapped[str] = mapped_column(String(200))
    location: Mapped[str] = mapped_column(String(200), default="Asosiy kirish")
    direction: Mapped[str] = mapped_column(String(10), default="in")
    is_active: Mapped[bool] = mapped_column(default=True)
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), server_default=func.now())

    school: Mapped["School"] = relationship(back_populates="devices")


from app.models.school import School # noqa: E402
```

app/models/attendance.py — Davomat modeli

app/models/attendance.py

```
from datetime import datetime

from sqlalchemy import DateTime, ForeignKey, String, func
from sqlalchemy.orm import Mapped, mapped_column

from app.database import Base


class AttendanceEvent(Base):
    __tablename__ = "attendance"

    id: Mapped[int] = mapped_column(primary_key=True)
    student_id: Mapped[int] = mapped_column(ForeignKey("students.id"), index=True)
    device_id: Mapped[int | None] = mapped_column(ForeignKey("devices.id"))
    direction: Mapped[str] = mapped_column(String(10))
    card_id: Mapped[str] = mapped_column(String(64))
    event_time: Mapped[datetime] = mapped_column(DateTime(timezone=True), index=True)
    notified: Mapped[bool] = mapped_column(default=False)
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), server_default=func.now())
```

app/models/settings.py — Tizim sozlamalari

app/models/settings.py

```
from datetime import datetime

from sqlalchemy import DateTime, Integer, String, Text, func
from sqlalchemy.orm import Mapped, mapped_column

from app.database import Base


class AppSettings(Base):
    __tablename__ = "settings"

    id: Mapped[int] = mapped_column(primary_key=True, default=1)
    trial_days: Mapped[int] = mapped_column(Integer, default=3)
    subscription_price: Mapped[int] = mapped_column(Integer, default=30000)
    subscription_period_days: Mapped[int] = mapped_column(Integer, default=30)
    currency: Mapped[str] = mapped_column(String(10), default="UZS")
    payment_card_number: Mapped[str | None] = mapped_column(String(30))
    payment_card_holder: Mapped[str | None] = mapped_column(String(100))
    payment_click_link: Mapped[str | None] = mapped_column(Text)
    payment_info: Mapped[str | None] = mapped_column(Text, default="To'lovdan keyin admin obunani faollashtiradi.")
    updated_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), server_default=func.now())
```

app/models/payment.py — To'lov buyurtmasi

app/models/payment.py

```
from datetime import datetime

from sqlalchemy import DateTime, ForeignKey, Integer, String, func
from sqlalchemy.orm import Mapped, mapped_column

from app.database import Base


class PaymentOrder(Base):
    __tablename__ = "payments"

    id: Mapped[int] = mapped_column(primary_key=True)
    parent_id: Mapped[int] = mapped_column(ForeignKey("parents.id"), index=True)
    amount: Mapped[int] = mapped_column(Integer)
    currency: Mapped[str] = mapped_column(String(10), default="UZS")
    status: Mapped[str] = mapped_column(String(20), default="pending")
    external_id: Mapped[str | None] = mapped_column(String(100))
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), server_default=func.now())
    paid_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True))
```

app/schemas/__init__.py — API sxemalari (qism)

app/schemas/__init__.py

```

from datetime import datetime
from enum import Enum

from pydantic import BaseModel, Field


class Direction(str, Enum):
    IN = "in"
    OUT = "out"


# --- Turniket ---
class TurnstileEvent(BaseModel):
    card_id: str
    device_code: str
    direction: Direction
    event_time: datetime | None = None


class TurnstileResponse(BaseModel):
    success: bool
    message: str
    event_id: int | None = None
    student_name: str | None = None
    notifications_sent: int = 0


# --- Maktab ---
class SchoolCreate(BaseModel):
    name: str
    address: str | None = None


class SchoolOut(BaseModel):
    id: int
    name: str
    address: str | None
    model_config = {"from_attributes": True}


# --- O'quvchi ---
class StudentCreate(BaseModel):
    school_id: int
    first_name: str
    last_name: str
    class_name: str
    card_id: str


class StudentUpdate(BaseModel):
    first_name: str | None = None
    last_name: str | None = None
    class_name: str | None = None
    card_id: str | None = None
    school_id: int | None = None
    is_active: bool | None = None


class StudentOut(BaseModel):

```

```

id: int
school_id: int
first_name: str
last_name: str
class_name: str
card_id: str
is_active: bool
model_config = {"from_attributes": True}

```

```

# --- Ota-ona ---

```

```

class ParentCreate(BaseModel):
    full_name: str
    phone: str | None = None
    student_ids: list[int] = Field(default_factory=list)

```

```

class ParentUpdate(BaseModel):
    full_name: str | None = None
    phone: str | None = None
    student_ids: list[int] | None = None
    is_active: bool | None = None

```

```

class ParentOut(BaseModel):
    id: int
    full_name: str
    phone: str | None
    telegram_chat_id: int | None
    is_active: bool
    subscription_status: str = "Kutilmoqda"
    model_config = {"from_attributes": True}

```

```

# --- Turniket qurilma ---

```

```

class DeviceCreate(BaseModel):
    school_id: int
    device_code: str
    name: str
    location: str = "Asosiy kirish"
    direction: Direction = Direction.IN

```

```

class DeviceOut(BaseModel):
    id: int
    school_id: int
    device_code: str
    name: str
    location: str
    direction: str
    is_active: bool
    model_config = {"from_attributes": True}

```

```

# --- Davomat ---

```

```

class AttendanceOut(BaseModel):
    id: int
    student_id: int
    direction: str
    card_id: str
    event_time: datetime
    notified: bool
    model_config = {"from_attributes": True}

```

```
# --- Obuna ---
class SettingsOut(BaseModel):
    trial_days: int
    subscription_price: int
    subscription_period_days: int
    currency: str
    payment_card_number: str | None = None
    payment_card_holder: str | None = None
    payment_click_link: str | None = None
    payment_info: str | None
    model_config = {"from_attributes": True}

class SettingsUpdate(BaseModel):
    trial_days: int | None = None
    subscription_price: int | None = None
    subscription_period_days: int | None = None
    currency: str | None = None
    payment_card_number: str | None = None
    payment_card_holder: str | None = None
    payment_click_link: str | None = None
    payment_info: str | None = None

class ActivateSub(BaseModel):
    months: int = Field(default=1, ge=1, le=12)
```

app/api/webhooks.py — Turniket webhook

app/api/webhooks.py

```

from fastapi import APIRouter, Depends, Header, HTTPException
from sqlalchemy.ext.asyncio import AsyncSession

from app.config import settings
from app.database import get_db
from app.schemas import TurnstileEvent, TurnstileResponse
from app.services.attendance import AttendanceService

router = APIRouter(prefix="/webhooks", tags=["Turniket"])

@router.post("/turnstile", response_model=TurnstileResponse)
async def turnstile(ev: TurnstileEvent, x: str = Header(..., alias="X-Webhook-Secret"), db: AsyncSession =
Depends(get_db)):
    if x != settings.webhook_secret:
        raise HTTPException(401, "Noto'g'ri kalit")
    svc = AttendanceService()
    att, msg, sent, name = await svc.process(db, ev)
    if not att:
        return TurnstileResponse(success=False, message=msg)
    return TurnstileResponse(success=True, message=msg, event_id=att.id, student_name=name,
notifications_sent=sent)

```


app/services/attendance.py — Davomat servisi

app/services/attendance.py

```

from datetime import datetime, timezone

from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy.orm import selectinload

from app.models.attendance import AttendanceEvent
from app.models.device import Device
from app.models.parent import Parent, StudentParent
from app.models.student import Student
from app.schemas import TurnstileEvent
from app.services.subscription import SubscriptionService
from app.services.telegram import TelegramService

class AttendanceService:
    def __init__(self) -> None:
        self.tg = TelegramService()

    async def process(self, db: AsyncSession, ev: TurnstileEvent) -> tuple:
        r = await db.execute(
            select(Student).options(selectinload(Student.school))
            .where(Student.card_id == ev.card_id, Student.is_active.is_(True))
        )
        student = r.scalar_one_or_none()
        if not student:
            return None, f"Karta topilmadi: {ev.card_id}", 0, None

        r = await db.execute(select(Device).where(Device.device_code == ev.device_code,
        Device.is_active.is_(True)))
        device = r.scalar_one_or_none()
        direction = device.direction if device and device.direction in ("in", "out") else ev.direction.value

        att = AttendanceEvent(
            student_id=student.id, device_id=device.id if device else None,
            direction=direction, card_id=ev.card_id,
            event_time=ev.event_time or datetime.now(timezone.utc),
        )
        db.add(att)
        await db.flush()

        settings = await SubscriptionService.get_settings(db)
        r = await db.execute(
            select(Parent).join(StudentParent).where(
                StudentParent.student_id == student.id,
                Parent.is_active.is_(True), Parent.telegram_chat_id.is_not(None),
            )
        )
        parents = list(r.scalars().all())
        msg = self.tg.attendance_msg(student, direction, att.event_time, device)
        sent = 0
        for p in parents:
            if SubscriptionService.status(p, settings)["notify"]:
                if await self.tg.send(p.telegram_chat_id, msg): # type: ignore
                    sent += 1
        att.notified = sent > 0
        await db.commit()
        name = f"{student.first_name} {student.last_name}"
        return att, f"{name} - {direction} qayd etildi", sent, name

```

app/services/telegram.py — Telegram xabar servisi

app/services/telegram.py

```

from datetime import datetime
from zoneinfo import ZoneInfo

from app.config import settings
from app.models.device import Device
from app.models.student import Student

class TelegramService:
    def __init__(self) -> None:
        self._bot = None

    def _bot_instance(self):
        if not settings.telegram_bot_token:
            return None
        if not self._bot:
            from telegram import Bot
            self._bot = Bot(token=settings.telegram_bot_token)
        return self._bot

    async def send(self, chat_id: int, text: str) -> bool:
        bot = self._bot_instance()
        if not bot:
            return False
        try:
            await bot.send_message(chat_id=chat_id, text=text, parse_mode="HTML")
            return True
        except Exception:
            return False

    def attendance_msg(self, student: Student, direction: str, event_time: datetime, device: Device | None) -> str:
        tz = ZoneInfo(settings.timezone)
        t = event_time.astimezone(tz)
        if direction == "in":
            action, emoji = "maktabga <b>kirdi</b>", ""
        else:
            action, emoji = "maktabdan <b>chiqdi</b>", ""
        loc = device.location if device else "Asosiy kirish"
        school = student.school.name if student.school else "Maktab"
        return (
            f"{emoji} <b>Farzandingiz haqida xabar</b>\n\n"
            f"<b>{student.first_name} {student.last_name}</b>\n"
            f"Sinf: {student.class_name}\n"
            f"{school}\n\n"
            f"{t.strftime('%H:%M')} ({t.strftime('%d.%m.%Y')})\n"
            f"{loc}\n"
            f"Farzandingiz {action}."
        )

```

app/services/subscription.py — Obuna servisi

app/services/subscription.py

```

from datetime import datetime, timedelta, timezone

from sqlalchemy.ext.asyncio import AsyncSession

from app.models.parent import Parent
from app.models.settings import AppSettings


class SubscriptionService:
    @staticmethod
    async def get_settings(db: AsyncSession) -> AppSettings:
        s = await db.get(AppSettings, 1)
        if not s:
            s = AppSettings(id=1)
            db.add(s)
            await db.commit()
            await db.refresh(s)
        return s

    @staticmethod
    def status(parent: Parent, settings: AppSettings) -> dict:
        now = datetime.now(timezone.utc)
        if not parent.telegram_chat_id:
            return {"code": "waiting", "label": "Kutilmoqda", "notify": False}
        if parent.subscription_until and parent.subscription_until > now:
            d = (parent.subscription_until - now).days
            return {"code": "active", "label": f"Obuna faol ({d} kun)", "notify": True}
        if parent.bot_registered_at:
            end = parent.bot_registered_at + timedelta(days=settings.trial_days)
            if now < end:
                d = (end - now).days + 1
                return {"code": "trial", "label": f"Bepul sinov ({d} kun)", "notify": True}
            return {"code": "expired", "label": "Obuna tugagan", "notify": False}
        return {"code": "waiting", "label": "Kutilmoqda", "notify": False}

    @staticmethod
    async def activate(db: AsyncSession, parent: Parent, settings: AppSettings, months: int = 1) -> None:
        now = datetime.now(timezone.utc)
        period = timedelta(days=settings.subscription_period_days * months)
        base = parent.subscription_until if parent.subscription_until and parent.subscription_until > now else
now
        parent.subscription_until = base + period
        if not parent.bot_registered_at:
            parent.bot_registered_at = now
        await db.commit()

```

app/services/payment.py — To'lov servisi

app/services/payment.py

```

import hashlib
import logging
from datetime import datetime, timezone

from sqlalchemy.ext.asyncio import AsyncSession

from app.config import settings
from app.models.parent import Parent
from app.models.payment import PaymentOrder
from app.models.settings import AppSettings
from app.services.subscription import SubscriptionService
from app.services.telegram import TelegramService

logger = logging.getLogger(__name__)

class PaymentService:
    def __init__(self) -> None:
        self.tg = TelegramService()

    async def create_order(self, db: AsyncSession, parent: Parent, s: AppSettings) -> PaymentOrder:
        order = PaymentOrder(parent_id=parent.id, amount=s.subscription_price, currency=s.currency)
        db.add(order)
        await db.commit()
        await db.refresh(order)
        return order

    def pay_url(self, order: PaymentOrder, cfg: AppSettings | None = None) -> str:
        if cfg and cfg.payment_click_link:
            return cfg.payment_click_link.strip()
        if settings.click_service_id and settings.click_merchant_id:
            return (
                f"https://my.click.uz/services/pay?service_id={settings.click_service_id}"
                f"&merchant_id={settings.click_merchant_id}&amount={order.amount}"
                f"&transaction_param={order.id}"
            )
        return f"{settings.app_base_url.rstrip('/')}/pay/{order.id}"

    def pay_message(self, order: PaymentOrder, cfg: AppSettings) -> str:
        lines = [
            f" <b>Obuna to'lovi</b>",
            f"",
            f" Summa: <b>{order.amount:,} {cfg.currency}</b>",
            f" Buyurtma: <b>#{order.id}</b>",
        ]
        if cfg.payment_card_number:
            holder = f"\n {cfg.payment_card_holder}" if cfg.payment_card_holder else ""
            lines += [f"", f" <b>Karta raqami:</b>", f"<code>{cfg.payment_card_number}</code>{holder}"]
        if cfg.payment_click_link:
            lines += [f"", f" Quyidagi tugma orqali <b>Click</b> da to'lang."]
        elif cfg.payment_card_number:
            lines += [f"", f" To'lovdan keyin chek skrinshotini maktab adminiga yuboring."]
        if cfg.payment_info:
            lines += [f"", f" {cfg.payment_info}"]
        return "\n".join(lines)

    async def complete(self, db: AsyncSession, order: PaymentOrder, ext_id: str | None = None) -> None:
        if order.status == "paid":
            return

```

```

order.status = "paid"
order.paid_at = datetime.now(timezone.utc)
order.external_id = ext_id
parent = await db.get(Parent, order.parent_id)
if not parent:
    return
s = await SubscriptionService.get_settings(db)
await SubscriptionService.activate(db, parent, s)
await db.refresh(parent)
if parent.telegram_chat_id:
    st = SubscriptionService.status(parent, s)
    await self.tg.send(
        parent.telegram_chat_id,
        f" <b>To'lov qabul qilindi!</b>\n\n"
        f" {order.amount:}, {order.currency}\n"
        f" {st['label']}\n\nXabarlar davom etadi.",
    )

def click_sign_ok(self, click_id, svc_id, merch_id, amount, action, sign_time, sign_string) -> bool:
    if not settings.click_secret_key:
        return settings.payment_demo_mode
    raw = f"{click_id}{svc_id}{settings.click_secret_key}{merch_id}{amount}{action}{sign_time}"
    return hashlib.md5(raw.encode()).hexdigest() == sign_string

async def click_webhook(self, db: AsyncSession, d: dict) -> dict:
    action = int(d.get("action", -1))
    merch_id = str(d.get("merchant_trans_id", ""))
    try:
        oid = int(merch_id)
    except ValueError:
        return {"error": -8, "error_note": "Noto'g'ri ID"}
    order = await db.get(PaymentOrder, oid)
    if not order:
        return {"error": -5, "error_note": "Topilmadi"}
    if not self.click_sign_ok(
        str(d.get("click_trans_id", "")), str(d.get("service_id", "")),
        merch_id, str(d.get("amount", "")), str(action),
        str(d.get("sign_time", "")), str(d.get("sign_string", "")),
    ):
        return {"error": -1, "error_note": "Imzo xato"}
    if action == 0:
        return {"click_trans_id": d.get("click_trans_id"), "merchant_trans_id": merch_id,
            "merchant_prepare_id": order.id, "error": 0, "error_note": "Success"}
    if action == 1:
        await self.complete(db, order, str(d.get("click_trans_id", "")))
        return {"click_trans_id": d.get("click_trans_id"), "merchant_trans_id": merch_id,
            "merchant_confirm_id": order.id, "error": 0, "error_note": "Success"}
    return {"error": -3, "error_note": "Xato action"}

```

app/bot/handlers.py — Telegram bot buyruqlari

app/bot/handlers.py

```

import logging
from datetime import datetime, timezone

from sqlalchemy import select
from sqlalchemy.orm import selectinload
from telegram import InlineKeyboardButton, InlineKeyboardMarkup, Update
from telegram.ext import Application, CommandHandler, ContextTypes

from app.config import settings
from app.database import SessionLocal
from app.models.parent import Parent, StudentParent
from app.models.student import Student
from app.services.payment import PaymentService
from app.services.subscription import SubscriptionService

log = logging.getLogger(__name__)

async def start(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text(
        "<b>Assalomu alaykum!</b>\n\n"
        "Farzandingiz maktabga kirishi/chiqishini xabar qilaman.\n\n"
        "<b>Buyruqlar:</b>\n"
        "/register &lt;id> – Ro'yxatdan o'tish\n"
        "/pay – Obuna to'lovi\n"
        "/status – Obuna holati\n"
        "/mystudents – Farzandlar\n"
        "/help – Yordam",
        parse_mode="HTML",
    )

async def help_cmd(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text(
        "1. Admin sizni qo'shadi va ID beradi\n"
        "2. <code>/register 5</code> yozing\n"
        "3. 3 kun bepul sinov\n"
        "4. Keyin <code>/pay</code> – to'lov, obuna avtomatik yoqiladi",
        parse_mode="HTML",
    )

async def register(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    if not ctx.args:
        return await update.message.reply_text("Misol: <code>/register 1</code>", parse_mode="HTML")
    try:
        pid = int(ctx.args[0])
    except ValueError:
        return await update.message.reply_text(" Noto'g'ri ID")
    chat = update.effective_user.id
    async with SessionLocal() as db:
        p = await db.get(Parent, pid)
        if not p:
            return await update.message.reply_text(" ID topilmadi")
        ex = await db.execute(select(Parent).where(Parent.telegram_chat_id == chat, Parent.id != pid))
        if ex.scalar_one_or_none():
            return await update.message.reply_text(" Bu hisob boshqa ota-onaga bog'langan")
        p.telegram_chat_id = chat
        p.telegram_username = update.effective_user.username

```

```

    if not p.bot_registered_at:
        p.bot_registered_at = datetime.now(timezone.utc)
    await db.commit()
    cfg = await SubscriptionService.get_settings(db)
    st = SubscriptionService.status(p, cfg)
    r = await db.execute(select(Student).join(StudentParent).where(StudentParent.parent_id == pid))
    kids = list(r.scalars().all())
    kids_txt = "\n".join(f"• {s.first_name} {s.last_name} ({s.class_name})" for s in kids) or "△ Farzand
bog'lanmagan"
    await update.message.reply_text(
        f" <b>Ro'yxatdan o'tdingiz!</b>\n\n {p.full_name}\n{kids_txt}\n\n"
        f" {cfg.trial_days} kun bepul: {st['label']}",
        parse_mode="HTML",
    )

async def status_cmd(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    chat = update.effective_user.id
    async with SessionLocal() as db:
        r = await db.execute(select(Parent).where(Parent.telegram_chat_id == chat))
        p = r.scalar_one_or_none()
        if not p:
            return await update.message.reply_text(" Avval /register qiling")
        cfg = await SubscriptionService.get_settings(db)
        st = SubscriptionService.status(p, cfg)
    await update.message.reply_text(
        f" <b>{p.full_name}</b>\nHolat: <b>{st['label']}</b>\n"
        f" {cfg.subscription_price:,} {cfg.currency} / {cfg.subscription_period_days} kun\n\n/pay – to'lov",
        parse_mode="HTML",
    )

async def pay_cmd(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    chat = update.effective_user.id
    ps = PaymentService()
    async with SessionLocal() as db:
        r = await db.execute(select(Parent).where(Parent.telegram_chat_id == chat))
        p = r.scalar_one_or_none()
        if not p:
            return await update.message.reply_text(" Avval /register qiling")
        cfg = await SubscriptionService.get_settings(db)
        st = SubscriptionService.status(p, cfg)
        if st["code"] == "active":
            return await update.message.reply_text(f" Obuna faol: {st['label']}")
        order = await ps.create_order(db, p, cfg)
        url = ps.pay_url(order, cfg)
        msg = ps.pay_message(order, cfg)
    buttons = []
    if url:
        buttons.append([InlineKeyboardButton(" Click orqali to'lash", url=url)])
    kb = InlineKeyboardMarkup(buttons) if buttons else None
    await update.message.reply_text(msg, parse_mode="HTML", reply_markup=kb)

async def mystudents(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    chat = update.effective_user.id
    async with SessionLocal() as db:
        r = await db.execute(select(Parent).where(Parent.telegram_chat_id == chat))
        p = r.scalar_one_or_none()
        if not p:
            return await update.message.reply_text(" /register qiling")
        r = await db.execute(select(Student).join(StudentParent).where(StudentParent.parent_id == p.id))
        kids = list(r.scalars().all())

```

```

if not kids:
    return await update.message.reply_text("Farzand topilmadi")
txt = "\n".join(f"• {s.first_name} {s.last_name} — {s.class_name} ({s.card_id})" for s in kids)
await update.message.reply_text(f" <b>Farzandlar:</b>\n\n{txt}", parse_mode="HTML")

def run_bot():
    if not settings.telegram_bot_token:
        raise ValueError("TELEGRAM_BOT_TOKEN kerak")
    log.info("Bot ishga tushmoqda...")
    app = Application.builder().token(settings.telegram_bot_token).build()
    for cmd, fn in [("start", start), ("help", help_cmd), ("register", register),
                    ("status", status_cmd), ("pay", pay_cmd), ("mystudents", mystudents)]:
        app.add_handler(CommandHandler(cmd, fn))
    app.run_polling()

```


app/api/payments.py — To'lov sahifalari

app/api/payments.py

```

from fastapi import APIRouter, Depends, Form, HTTPException, Request
from fastapi.responses import HTMLResponse
from sqlalchemy.ext.asyncio import AsyncSession

from app.config import settings
from app.database import get_db
from app.services.payment import PaymentService

router = APIRouter(tags=["To'lov"])
wh = APIRouter(prefix="/webhooks", tags=["To'lov"])

@wh.post("/payment/click")
async def click_hook(request: Request, db: AsyncSession = Depends(get_db)):
    return await PaymentService().click_webhook(db, dict(await request.form()))

@router.get("/pay/{oid}", response_class=HTMLResponse)
async def pay_page(oid: int, db: AsyncSession = Depends(get_db)):
    from app.models.payment import PaymentOrder
    order = await db.get(PaymentOrder, oid)
    if not order:
        return HTMLResponse("<h1>Topilmadi</h1>", 404)
    if order.status == "paid":
        return HTMLResponse("<h1> To'lov qilingan</h1><p>Botga qayting</p>")
    ps = PaymentService()
    from app.services.subscription import SubscriptionService as SS
    cfg = await SS.get_settings(db)
    url = ps.pay_url(order, cfg)
    demo = f'<form method="POST" action="/pay/{oid}/done" style="margin-top:20px">
        <button style="padding:15px
40px;background:#2563eb;color:#fff;border:none;border-radius:8px;font-size:16px;cursor:pointer">
        To'lov qildim</button></form>' if settings.payment_demo_mode else ""
    card = ""
    if cfg.payment_card_number:
        holder = f'<p> {cfg.payment_card_holder}</p>' if cfg.payment_card_holder else ""
        card = f'<p style="font-size:18px"> Karta: <b
style="letter-spacing:1px">{cfg.payment_card_number}</b></p>{holder}'
        show_click = cfg.payment_click_link or settings.click_service_id
        click = f'<a href="{url}" style="display:inline-block;padding:15px
40px;background:#00A651;color:#fff;text-decoration:none;border-radius:8px;margin:10px">Click orqali
to'lash</a>' if show_click else ""
        info = f'<p style="color:#666">{cfg.payment_info}</p>' if cfg.payment_info else ""
        return HTMLResponse(f'<!DOCTYPE html><html><head><meta charset="utf-8"><meta name="viewport"
content="width=device-width,initial-scale=1">
        <title>To'lov</title></head><body style="font-family:sans-serif;text-align:center;padding:40px">
        <h1> Obuna to'lovi</h1><p style="font-size:28px;font-weight:bold">{order.amount:,} {order.currency}</p>
        {card}{click}{demo}{info}<p style="color:#666;margin-top:20px">To'lovdan keyin obuna avtomatik
yoqiladi</p></body></html>'")

@router.post("/pay/{oid}/done", response_class=HTMLResponse)
async def pay_done(oid: int, db: AsyncSession = Depends(get_db)):
    from app.models.payment import PaymentOrder
    order = await db.get(PaymentOrder, oid)
    if not order:
        raise HTTPException(404)
    await PaymentService().complete(db, order, f"demo-{oid}")
    return HTMLResponse("<h1> Obuna faollashtirildi!</h1><p>Telegram botga qayting</p>")

```

app/api/admin.py — Admin API (qism)

app/api/admin.py

```

from fastapi import APIRouter, Depends, Header, HTTPException
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession

from app.config import settings
from app.database import get_db
from app.models.device import Device
from app.models.parent import Parent, StudentParent
from app.models.school import School
from app.models.settings import AppSettings
from app.models.student import Student
from app.models.attendance import AttendanceEvent
from app.schemas import *
from app.services.subscription import SubscriptionService

router = APIRouter(prefix="/api", tags=["Admin"])

def admin(x: str = Header(..., alias="X-Admin-Key")):
    if x != settings.admin_api_key:
        raise HTTPException(401, "Noto'g'ri kalit")

# ---- MAKTAB ----
@router.get("/schools", response_model=list[SchoolOut], dependencies=[Depends(admin)])
async def schools(db: AsyncSession = Depends(get_db)):
    r = await db.execute(select(School).order_by(School.id))
    return list(r.scalars().all())

@router.post("/schools", response_model=SchoolOut, dependencies=[Depends(admin)])
async def create_school(d: SchoolCreate, db: AsyncSession = Depends(get_db)):
    s = School(**d.model_dump())
    db.add(s)
    await db.commit()
    await db.refresh(s)
    return s

# ---- O'QUVCHI ----
@router.get("/students", response_model=list[StudentOut], dependencies=[Depends(admin)])
async def students(db: AsyncSession = Depends(get_db)):
    r = await db.execute(select(Student).order_by(Student.id))
    return list(r.scalars().all())

@router.post("/students", response_model=StudentOut, dependencies=[Depends(admin)])
async def create_student(d: StudentCreate, db: AsyncSession = Depends(get_db)):
    if not await db.get(School, d.school_id):
        raise HTTPException(404, "Maktab topilmadi")
    ex = await db.execute(select(Student).where(Student.card_id == d.card_id))
    if ex.scalar_one_or_none():
        raise HTTPException(400, "Karta mavjud")
    s = Student(**d.model_dump())
    db.add(s)
    await db.commit()
    await db.refresh(s)
    return s

```

```

@router.put("/students/{sid}", response_model=StudentOut, dependencies=[Depends(admin)])
async def update_student(sid: int, d: StudentUpdate, db: AsyncSession = Depends(get_db)):
    s = await db.get(Student, sid)
    if not s:
        raise HTTPException(404, "Topilmadi")
    for k, v in d.model_dump(exclude_unset=True).items():
        setattr(s, k, v)
    await db.commit()
    await db.refresh(s)
    return s

@router.delete("/students/{sid}", dependencies=[Depends(admin)])
async def delete_student(sid: int, db: AsyncSession = Depends(get_db)):
    s = await db.get(Student, sid)
    if not s:
        raise HTTPException(404, "Topilmadi")
    for l in (await db.execute(select(StudentParent).where(StudentParent.student_id == sid))).scalars():
        await db.delete(l)
    await db.delete(s)
    await db.commit()
    return {"ok": True}

# ---- OTA-ONA ----
@router.get("/parents", response_model=list[ParentOut], dependencies=[Depends(admin)])
async def parents(db: AsyncSession = Depends(get_db)):
    cfg = await SubscriptionService.get_settings(db)
    r = await db.execute(select(Parent).order_by(Parent.id))
    out = []
    for p in r.scalars().all():
        out.append(ParentOut(
            id=p.id, full_name=p.full_name, phone=p.phone,
            telegram_chat_id=p.telegram_chat_id, is_active=p.is_active,
            subscription_status=SubscriptionService.status(p, cfg)["label"],
        ))
    return out

@router.post("/parents", response_model=ParentOut, dependencies=[Depends(admin)])
async def create_parent(d: ParentCreate, db: AsyncSession = Depends(get_db)):
    p = Parent(full_name=d.full_name, phone=d.phone)
    db.add(p)
    await db.flush()
    for sid in d.student_ids:
        if await db.get(Student, sid):
            db.add(StudentParent(student_id=sid, parent_id=p.id))
    await db.commit()
    await db.refresh(p)
    cfg = await SubscriptionService.get_settings(db)
    return ParentOut(id=p.id, full_name=p.full_name, phone=p.phone,
        telegram_chat_id=p.telegram_chat_id, is_active=p.is_active,
        subscription_status=SubscriptionService.status(p, cfg)["label"])

@router.put("/parents/{pid}", response_model=ParentOut, dependencies=[Depends(admin)])
async def update_parent(pid: int, d: ParentUpdate, db: AsyncSession = Depends(get_db)):
    p = await db.get(Parent, pid)
    if not p:
        raise HTTPException(404, "Topilmadi")
    data = d.model_dump(exclude_unset=True)
    ids = data.pop("student_ids", None)

```

```

for k, v in data.items():
    setattr(p, k, v)
if ids is not None:
    for l in (await db.execute(select(StudentParent).where(StudentParent.parent_id == pid))).scalars():
        await db.delete(l)
    for sid in ids:
        if await db.get(Student, sid):
            db.add(StudentParent(student_id=sid, parent_id=pid))
await db.commit()
await db.refresh(p)
cfg = await SubscriptionService.get_settings(db)
return ParentOut(id=p.id, full_name=p.full_name, phone=p.phone,
                  telegram_chat_id=p.telegram_chat_id, is_active=p.is_active,
                  subscription_status=SubscriptionService.status(p, cfg)["label"])

@router.delete("/parents/{pid}", dependencies=[Depends(admin)])
async def delete_parent(pid: int, db: AsyncSession = Depends(get_db)):
    p = await db.get(Parent, pid)
    if not p:
        raise HTTPException(404, "Topilmadi")
    for l in (await db.execute(select(StudentParent).where(StudentParent.parent_id == pid))).scalars():
        await db.delete(l)
    await db.delete(p)
    await db.commit()
    return {"ok": True}

@router.post("/parents/{pid}/subscribe", dependencies=[Depends(admin)])
async def activate_sub(pid: int, d: ActivateSub, db: AsyncSession = Depends(get_db)):
    p = await db.get(Parent, pid)
    if not p:
        raise HTTPException(404, "Topilmadi")
    cfg = await SubscriptionService.get_settings(db)
    await SubscriptionService.activate(db, p, cfg, d.months)
    return {"ok": True, "status": SubscriptionService.status(p, cfg)["label"]}

# ---- TURNIKET ----
@router.get("/devices", response_model=list[DeviceOut], dependencies=[Depends(admin)])
async def devices(db: AsyncSession = Depends(get_db)):
    r = await db.execute(select(Device).order_by(Device.id))
    return list(r.scalars().all())

@router.post("/devices", response_model=DeviceOut, dependencies=[Depends(admin)])
async def create_device(d: DeviceCreate, db: AsyncSession = Depends(get_db)):
    data = d.model_dump()
    data["direction"] = d.direction.value
    dev = Device(**data)
    db.add(dev)
    await db.commit()
    await db.refresh(dev)
    return dev

# ---- DAVOMAT ----
@router.get("/attendance", response_model=list[AttendanceOut], dependencies=[Depends(admin)])
async def attendance(db: AsyncSession = Depends(get_db)):
    r = await db.execute(select(AttendanceEvent).order_by(AttendanceEvent.event_time.desc()).limit(100))
    return list(r.scalars().all())

```

```
# ---- OBUNA SOZLAMALARI ----
@router.get("/settings", response_model=SettingsOut, dependencies=[Depends(admin)])
async def get_settings(db: AsyncSession = Depends(get_db)):
    return await SubscriptionService.get_settings(db)

@router.put("/settings", response_model=SettingsOut, dependencies=[Depends(admin)])
async def update_settings(d: SettingsUpdate, db: AsyncSession = Depends(get_db)):
    s = await SubscriptionService.get_settings(db)
    for k, v in d.model_dump(exclude_unset=True).items():
        setattr(s, k, v)
    await db.commit()
    await db.refresh(s)
    return s
```

app/api/admin.py — Admin API (davomi)

app/api/admin.py (100-qatordan)

```

async def create_parent(d: ParentCreate, db: AsyncSession = Depends(get_db)):
    p = Parent(full_name=d.full_name, phone=d.phone)
    db.add(p)
    await db.flush()
    for sid in d.student_ids:
        if await db.get(Student, sid):
            db.add(StudentParent(student_id=sid, parent_id=p.id))
    await db.commit()
    await db.refresh(p)
    cfg = await SubscriptionService.get_settings(db)
    return ParentOut(id=p.id, full_name=p.full_name, phone=p.phone,
                     telegram_chat_id=p.telegram_chat_id, is_active=p.is_active,
                     subscription_status=SubscriptionService.status(p, cfg)["label"])

@router.put("/parents/{pid}", response_model=ParentOut, dependencies=[Depends(admin)])
async def update_parent(pid: int, d: ParentUpdate, db: AsyncSession = Depends(get_db)):
    p = await db.get(Parent, pid)
    if not p:
        raise HTTPException(404, "Topilmadi")
    data = d.model_dump(exclude_unset=True)
    ids = data.pop("student_ids", None)
    for k, v in data.items():
        setattr(p, k, v)
    if ids is not None:
        for l in (await db.execute(select(StudentParent).where(StudentParent.parent_id == pid))).scalars():
            await db.delete(l)
        for sid in ids:
            if await db.get(Student, sid):
                db.add(StudentParent(student_id=sid, parent_id=pid))
    await db.commit()
    await db.refresh(p)
    cfg = await SubscriptionService.get_settings(db)
    return ParentOut(id=p.id, full_name=p.full_name, phone=p.phone,
                     telegram_chat_id=p.telegram_chat_id, is_active=p.is_active,
                     subscription_status=SubscriptionService.status(p, cfg)["label"])

@router.delete("/parents/{pid}", dependencies=[Depends(admin)])
async def delete_parent(pid: int, db: AsyncSession = Depends(get_db)):
    p = await db.get(Parent, pid)
    if not p:
        raise HTTPException(404, "Topilmadi")
    for l in (await db.execute(select(StudentParent).where(StudentParent.parent_id == pid))).scalars():
        await db.delete(l)
    await db.delete(p)
    await db.commit()
    return {"ok": True}

@router.post("/parents/{pid}/subscribe", dependencies=[Depends(admin)])
async def activate_sub(pid: int, d: ActivateSub, db: AsyncSession = Depends(get_db)):
    p = await db.get(Parent, pid)
    if not p:
        raise HTTPException(404, "Topilmadi")
    cfg = await SubscriptionService.get_settings(db)
    await SubscriptionService.activate(db, p, cfg, d.months)
    return {"ok": True, "status": SubscriptionService.status(p, cfg)["label"]}

```

```

# ---- TURNIKET ----
@router.get("/devices", response_model=list[DeviceOut], dependencies=[Depends(admin)])
async def devices(db: AsyncSession = Depends(get_db)):
    r = await db.execute(select(Device).order_by(Device.id))
    return list(r.scalars().all())

@router.post("/devices", response_model=DeviceOut, dependencies=[Depends(admin)])
async def create_device(d: DeviceCreate, db: AsyncSession = Depends(get_db)):
    data = d.model_dump()
    data["direction"] = d.direction.value
    dev = Device(**data)
    db.add(dev)
    await db.commit()
    await db.refresh(dev)
    return dev

# ---- DAVOMAT ----
@router.get("/attendance", response_model=list[AttendanceOut], dependencies=[Depends(admin)])
async def attendance(db: AsyncSession = Depends(get_db)):
    r = await db.execute(select(AttendanceEvent).order_by(AttendanceEvent.event_time.desc()).limit(100))
    return list(r.scalars().all())

# ---- OBUNA SOZLAMALARI ----
@router.get("/settings", response_model=SettingsOut, dependencies=[Depends(admin)])
async def get_settings(db: AsyncSession = Depends(get_db)):
    return await SubscriptionService.get_settings(db)

@router.put("/settings", response_model=SettingsOut, dependencies=[Depends(admin)])
async def update_settings(d: SettingsUpdate, db: AsyncSession = Depends(get_db)):
    s = await SubscriptionService.get_settings(db)
    for k, v in d.model_dump(exclude_unset=True).items():
        setattr(s, k, v)
    await db.commit()
    await db.refresh(s)
    return s

```